

# СОФТУЕРНИ АРХИТЕКТУРИ

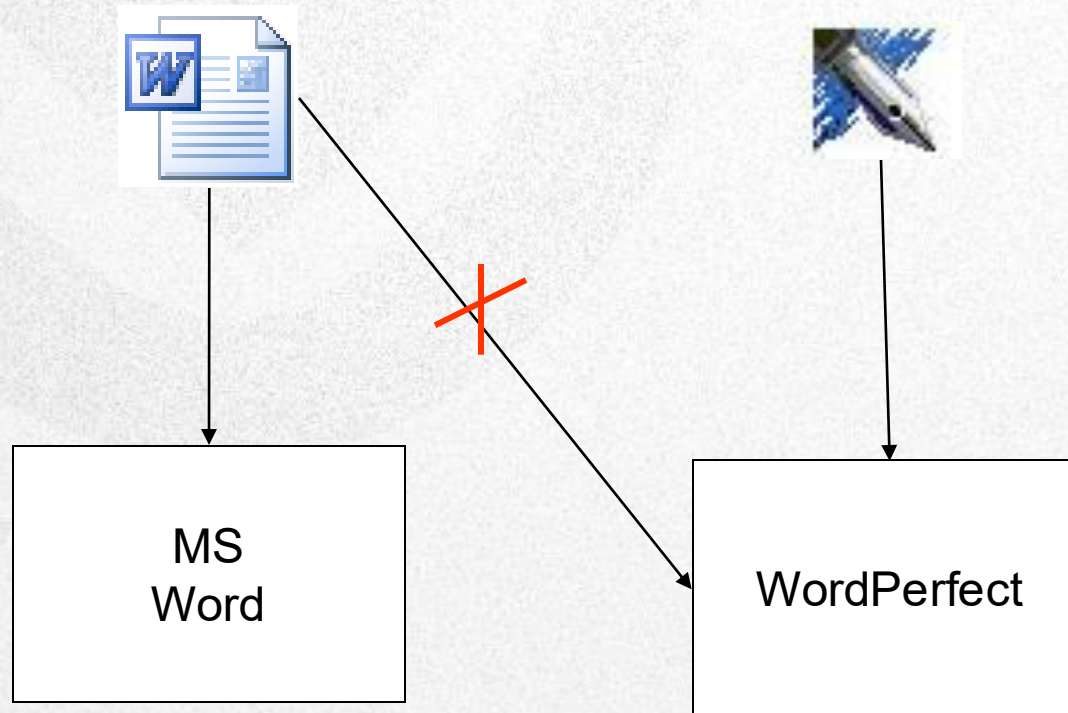
## ВЪВЕДЕНИЕ В XML



# 1. ДАННИ, ФАЙЛОВЕ, ТЕКСТ

- Видове файлове
  - Двоични файлове
    - Поток от битове
    - Обработват се много бързо
    - Ефикасни за съхраняване на метаданни
    - Недостатък: частни – обработват се от специални програми

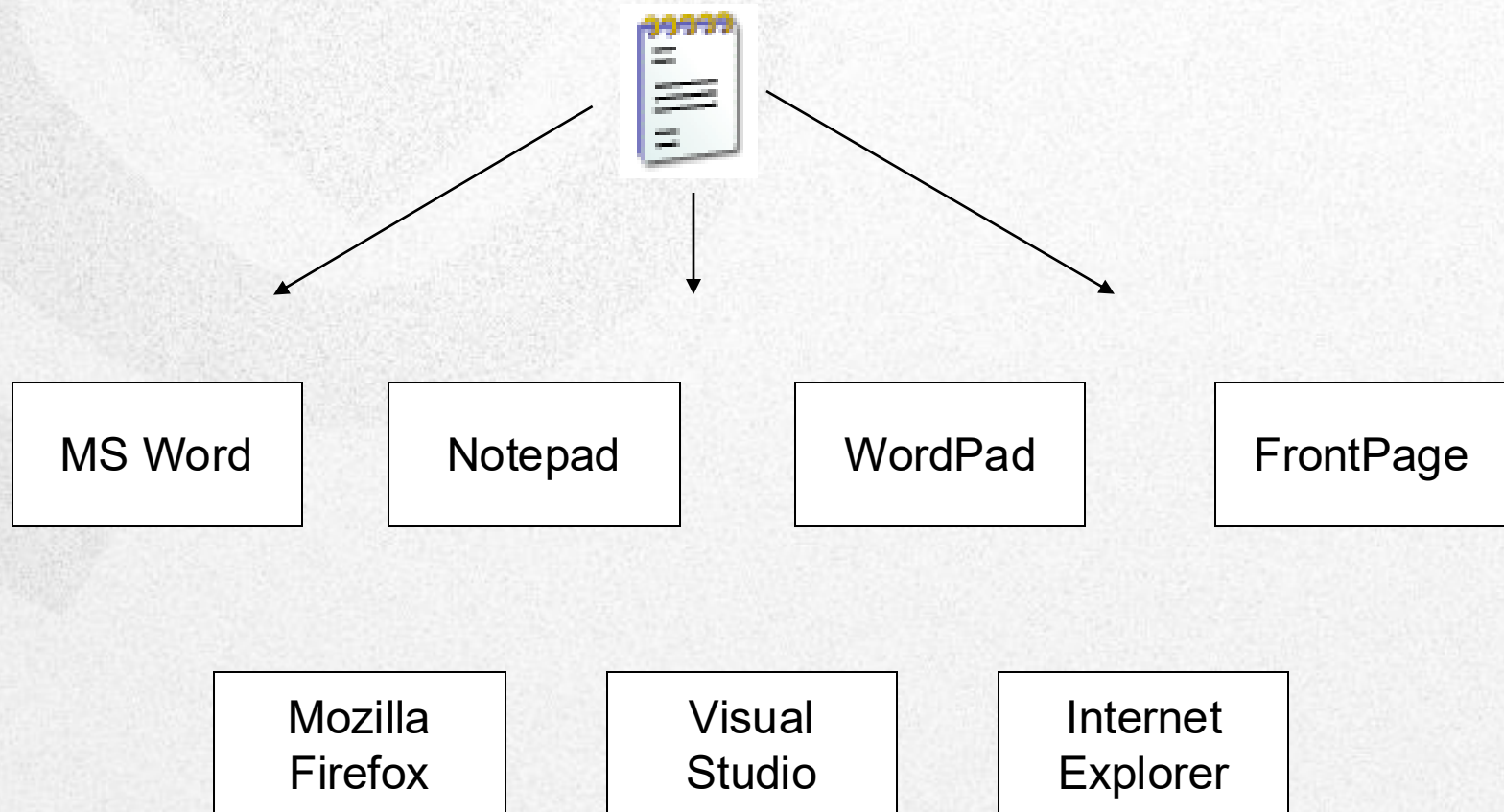
# ДАННИ, ФАЙЛОВЕ, ТЕКСТ



# ДАННИ, ФАЙЛОВЕ, ТЕКСТ

- Текстови файлове
  - Също поток от битове, но групирани по стандартен начин, така че да представят числа. Тези числа съответстват на знаци (кодировка).
  - Голям набор от приложения могат да ги четат (дори хора с помощта на текстов редактор)
  - Лесно споделяне на информацията

# ДАННИ, ФАЙЛОВЕ, ТЕКСТ



# ДАННИ, ФАЙЛОВЕ, ТЕКСТ

- Интернет – в началото почти изцяло текстово базиран, което допринася за експлозивното му разпространение
- Недостатък на текстовите файлове -трудно се добавят метаданни

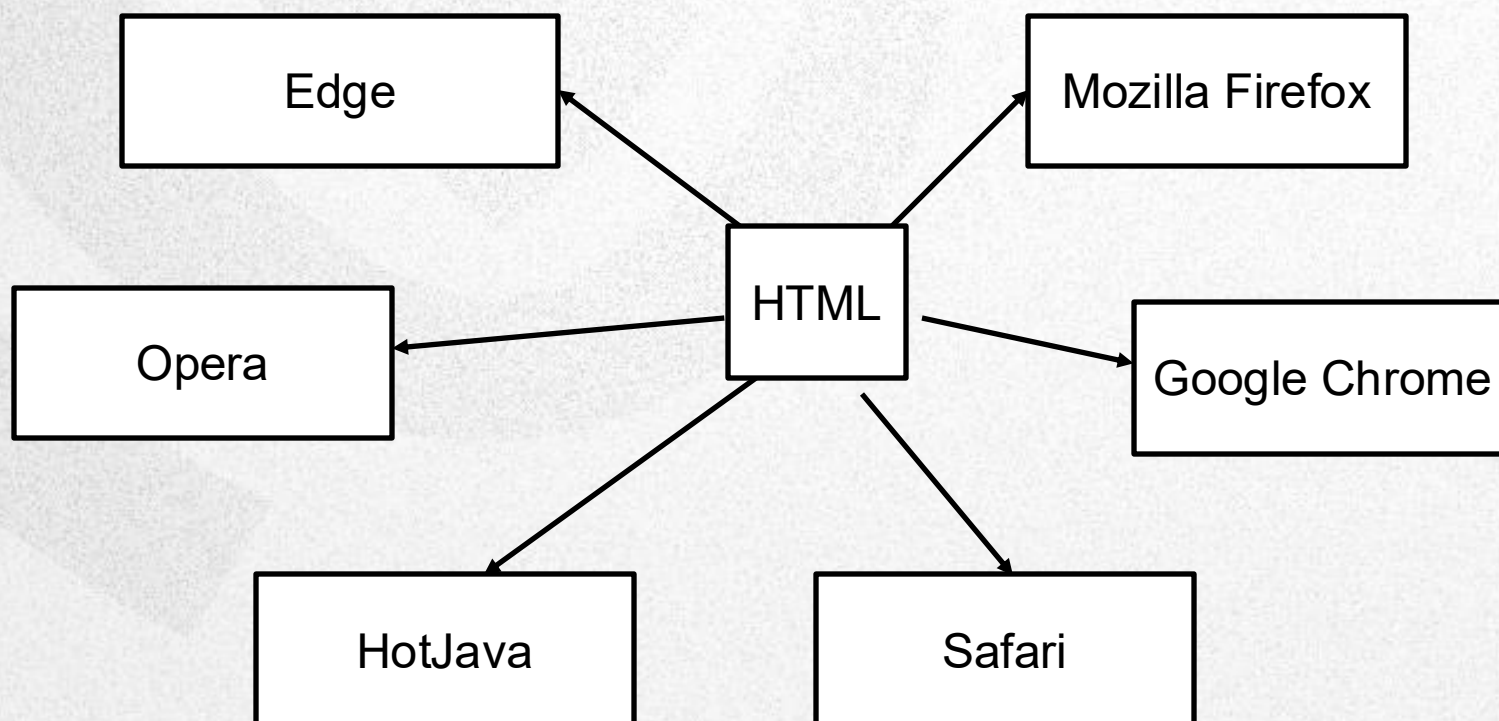
## 2. ИСТОРИЯ НА ЕЗИЦИТЕ ЗА ФОРМАТИРАНЕ

- SGML (Standard Generalized Markup Language)
  - текстово-базиран език за форматиране (markup) на данни (добавяне на метаданни) по самоописателен начин
  - универсален начин за форматиране на данни за произволна цел
  - прилаган предимно в огромни системи за управление на документи
  - в резултат на това SGML става много сложен (но с това идва и мощта)

# ИСТОРИЯ НА ЕЗИЦИТЕ ЗА ФОРМАТИРАНЕ

- HTML (HyperText Markup Language)
  - най-известното приложение на SGML
  - универсален език за форматиране
  - служи за представяне на информация и свързване на различни парчета информация
  - идеята е всеки HTML документ, да може да се представи във всяко приложение, което разбира HTML – т.е. браузър.

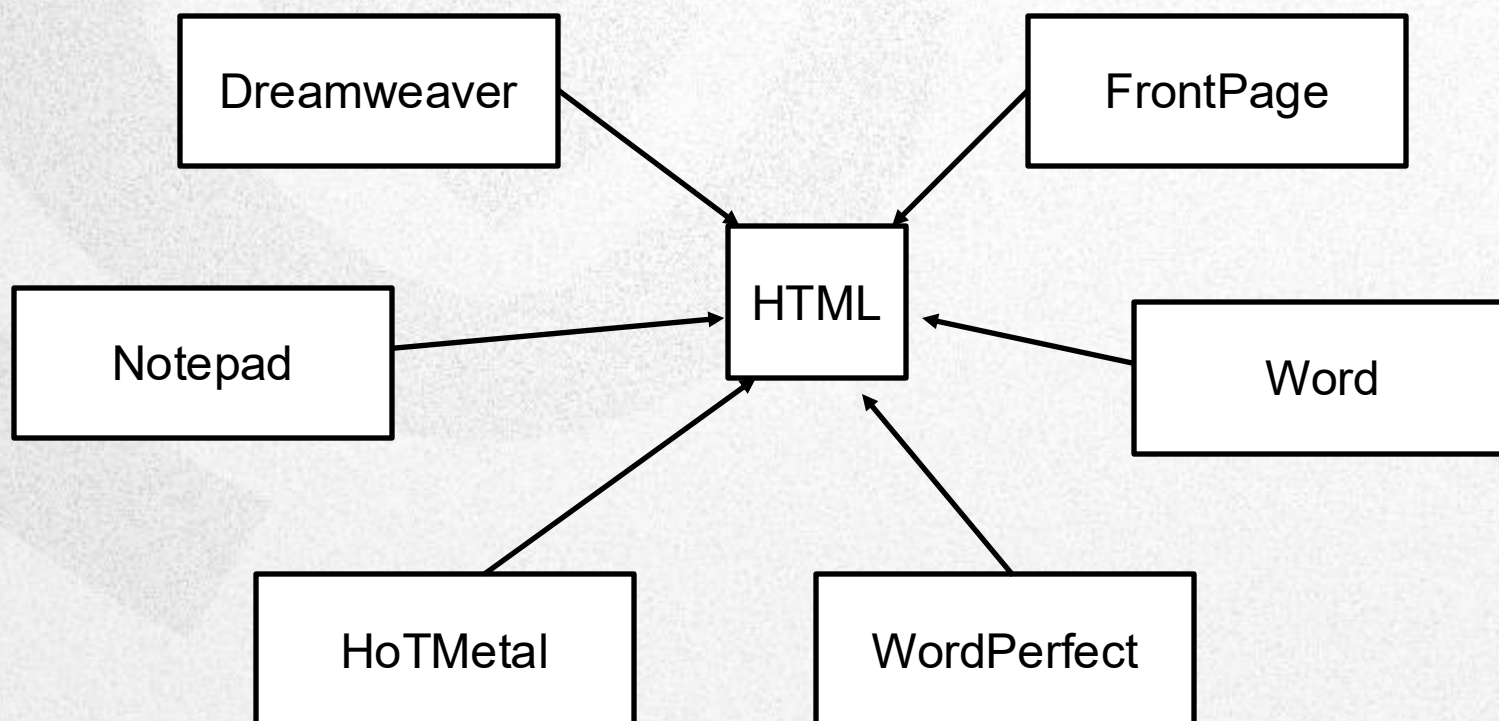
# ИСТОРИЯ НА ЕЗИЦИТЕ ЗА ФОРМАТИРАНЕ



# ИСТОРИЯ НА ЕЗИЦИТЕ ЗА ФОРМАТИРАНЕ

- Понеже HTML е текстово-базиран, то всеки може да създаде HTML страница с помощта на прост текстов редактор или с някоя от многобройните програми за редактиране на web страници

# ИСТОРИЯ НА ЕЗИЦИТЕ ЗА ФОРМАТИРАНЕ



# 3.КАКВО Е XML?

- SGML е прекалено сложен и не е пригоден за обмен на данни през web.
- HTML е много разпространен, но той също има ограничения – предназначен е само за показване на документи в браузър.

# КАКВО Е XML?

- XML – Extensible Markup Language
  - подмножество на SGML със същите цели (форматиране на произволен тип данни) от което са премахнати възможно най-много усложняващи неща
  - не е език, а стандарт за създаване на езици, които отговарят на XML критерии
  - XML описва синтаксиса, който трябва да се използва за създаване на собствени езици

# КАКВО Е XML?

- Пример: имаме данни за едно име и искаме да можем да споделяме тази информация и с други хора

Текстов файл

John Doe

HTML документ

```
<HTML>
  <HEAD>
    <TITLE>Име</TITLE>
  </HEAD>
  <BODY>
    <P>John Doe</P>
  </BODY>
</HTML>
```

# КАКВО Е XML?

XML документ

```
<name>  
  <first>John</first>  
  <last>Doe</last>  
</name>
```

# КАКВО Е XML?

- XML е самоописателен. От самото съдържание се разбира за какво се отнасят данните
- XML варианта е много по-голям от този на обикновения текстов файл:
  - XML увеличава значително размера на данните
  - малкия размер не е основна цел на XML

# КАКВО Е XML?

- Основна идея на XML: улесняване на създаването на софтуер, който извършва достъп до данните.
- Предимствата от лесен-за-писане код са много по-големи отколкото недостатъците от увеличаване на размера на данните за предаване.

## 4. КАКВО НИ ДАВА XML?

- Как по друг начин бихме могли да опишем данните?

John **Fitzgerald** Doe

John Fitzgerald **Johansen** Doe **the 3rd**

- Докато един човек може веднага да каже, че двете думи по средата съставят презимето, то е доста трудно тази логика да се представи в една програма.

# КАКВО НИ ДАВА XML?

- В повечето подобни случаи просто се дефинират по-рестриктивни правила от софтуерните разработчици – напр. ще се поддържа само едно презиме.
- Това води до структуриране на данните по безкрайно много начини, а с всеки нов начин идва и нова методология за извличането на необходимата информация (следват експерименти, тестове,...).

# КАКВО НИ ДАВА XML?

- Ако данните се променят, трябва да се промени и методологията (пак тестове и експерименти).
- С XML съществува стандартен начин за извличане на необходимата ни информация независимо от това как е структурирана.

# XML ПАРСЕРИ

- Парсер – програма, която може да чете XML синтаксиса и да извлича информацията.
- Използват се в приложенията. По този начин не се работи с XML директно.
- Вършат тежката работа.

# XML ПАРСЕРИ

```
<name>  
  <first>John</first>  
  <middle>Fitzgerald Johansen</middle>  
  <last>Doe</last>  
</name>
```

- Парсерът “ще каже”, че “има едно парче данни, наречено <middle> и то съдържа Fitzgerald Johansen”.
- Създателя на парсера не е знаел никакви правила за това къде свършва първото име и започва презимето.

# XML ПАРСЕРИ

- Езикът на който е написан XML кодът също няма никакво значение.
- Предимство: XML форматът от примера може да бъде разширяван без да се налага да се променя кода на приложението.
- По този начин новите приложения могат да разширят формата и старите да продължат да функционират без да се променят.

# ЗАЩО EXTENSIBLE?

- Имаме пълен контрол над създаването на XML документа
  - можем да групираме данните както ни е удобно `<name>John Fitzgerald Johansen Doe</name>`
  - могат да се създават данни, които да се използват само от едно конкретно приложение или да се споделят в Интернет
  - свободно могат да се структурират едни и същи данни по различен начин – подходящ за дадено приложение или цяла група приложения

# ПРИЛОЖЕНИЯ НА XML

- Scalable Vector Graphics (SVG) – XML речник за описание на двумерни графики
- MathML – XML речник за описание на математически операции като база за комуникация от тип машина с машина
- Chemical Markup Language (CML) – XML речник за управление на химическа информация.
- ...

# HTML И XML

- HTML е предназначен за специфично приложение – да доставя информация на хората (обикновено чрез web браузър)
  - има краен брой специфични конструкции (<p>, <ul>, <h2>, ...)
  - на теория всеки браузър ще разбере даден HTML документ, защото работи с краен брой тагове

# HTML И XML

- XML няма специфично приложение
  - всеки парсер ще извлече информацията от всеки XML документ
  - но не всяко приложение знае *какво означава тази информация*

# ЙЕРАРХИИ С ИНФОРМАЦИЯ

- Големи количества информация е добре да бъдат групирани в свързани подтеми (напр. организацията на дадена книга, данъчна декларация и др.)
- Това улеснява разбирането на информацията и достъпа до нея.

# ЙЕРАРХИИ В HTML

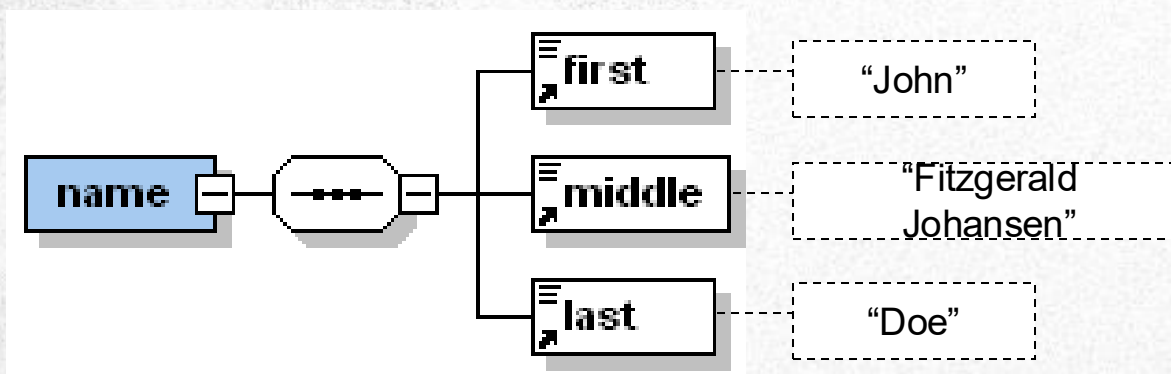
- При работа с HTML съществува Document Object Model (DOM)

```
alert(document.title);
```

# ЙЕРАРХИИ В XML

- XML групира информацията в йерархии.
- Взаимовръзките между елементите в документите са от тип **родител/наследник** или **брат(сестра)**.

# ЙЕРАРХИИ В XML



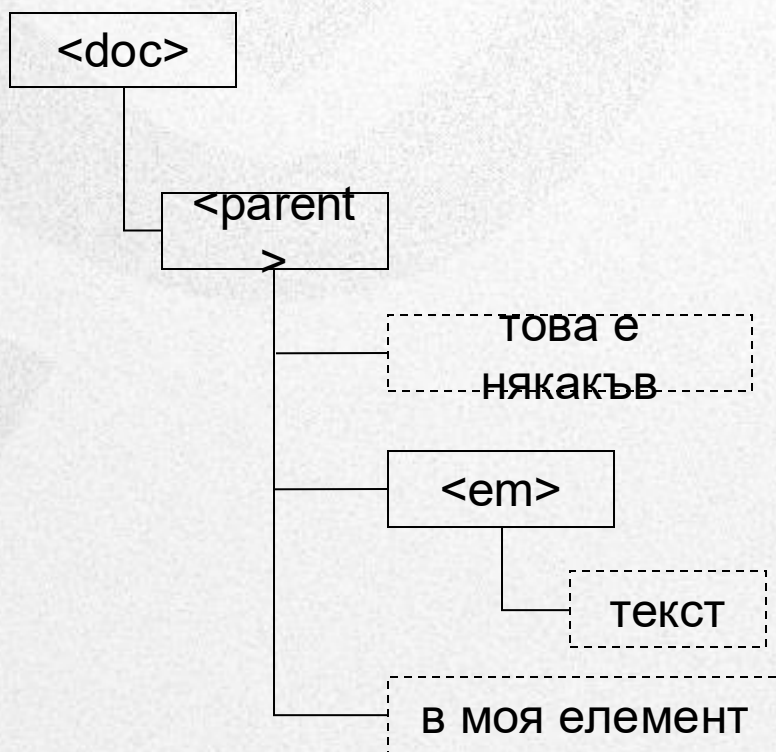
- Структурата се нарича **дърво**
- Всяка част от дървото, която съдържа наследници се нарича **клон**
- Частите без наследници са **листа**

# ЙЕРАРХИИ В XML

- <name> има за наследници само други елементи – **елементно съдържание**
- <first>, <middle>, <last> имат само текст за наследници – **просто съдържание**
- елементите, които съдържат и текст и други елементи – **смесено съдържание**

# ЙЕРАРХИИ В XML

```
<doc>  
  <parent>това е някакъв <em>текст</em> в моя елемент</parent>  
</doc>
```



# КАКВО Е ТИП НА ДОКУМЕНТА?

- Едно от големите достойнства на XML – може да се описва произволна информация
- Много гъвкав по отношение на структуриране на данните
- Можем да определим определен дизайн на информацията
- Може да създаваме специфични набори от елементи - речници

# ДОКУМЕНТЕН ТИП

- Специфичен тип документ, който е структуриран по определен начин за описание на специфичен тип информация
- Съществуват също така различни синтактични правила, които ни позволяват да дефинираме формално документите.

# КОМПОНЕНТИ НА XML (МНОЖЕСТВО ОТ СПЕЦИФИКАЦИИ):

XML – базова спецификация, която:

- описва синтаксиса, който трябва да следват XML документите и XML парсерите
- Другите средства, необходими за четене и писане на XML документи

# КОМПОНЕНТИ НА XML (МНОЖЕСТВО ОТ СПЕЦИФИКАЦИИ):

- DTD, XSD – шаблони за създаване на различни типове документи
- Namespaces – средства за различаване на различни речници
- XPath – език за заявки за обръщение към отделните части на един XML документ

# КОМПОНЕНТИ НА XML (МНОЖЕСТВО ОТ СПЕЦИФИКАЦИИ):

- CSS (Cascading Style Sheets) – подпомагат визуализацията на документите
- XPointer, XLink – за свързване на документи посредством хипервръзки
- DOM – документен обектен модел.

# ИЗПОЛЗВАНЕ НА XML

- Намалява натоварването на сървърите
- Съдържание на Web сайтовете
- Отдалечено извикване на процедури
- Електронна търговия

# СЪЗДАВАНЕ НА XML ДОКУМЕНТИ

Тагове – ключови думи заградени с < >

- Информацията в документа се разполага в различни тагове
- Така се улеснява разграничаването на същинските данни от използвания формат
- Различаваме – започващи и завършващи тагове

# ЕЛЕМЕНТИ:

- Цялата информацията от началото на започващия таг до края на завършващия таг
- Пример:
  - `<first>` - започващ таг
  - `</first>` - завършващ таг
  - `<first>Ivan</first>` - елемент

# ЕЛЕМЕНТИ:

- Текстът между започващия и завършващия таг – съдържание на елемента

# ПРАВИЛА ЗА ЕЛЕМЕНТИТЕ:

- Всеки започващ таг трябва да има завършващ таг
- Таговете не могат да се припокриват
- XML документите могат да имат само един главен елемент
- Имената трябва да съответстват на XML конвенциите за именуване
- Различават се големи и малки букви
- Запазват се интервалите в текстовете

# АТРИБУТИ

- Освен тагове и елементи XML документите могат да съдържат и атрибути
- Представяват прости двойки име/стойност – асоциират се с даден елемент
- Прикачват се към започващия таг

# ПРИМЕР

```
<name nickname="MrX">  
  <first>Ivan</first>  
  <last>Ivanov</last>  
</name>
```

# КОМЕНТАРИ

- Можем да вмъкваме текст, който не се интерпретира като част от документа
- Започват с <! - -
- Завършват с - - >

# ПРАЗНИ ЕЛЕМЕНТИ

- Понякога елементите не съдържат никакви данни
- Единственият случай, когато не се нуждаем от завършващ таг
- Пример: `<middle/>`

# XML ДЕКЛАРАЦИИ

- Понякога е удобно да можем да идентифицираме конкретния тип на даден документ
- За тази цел – използват се декларации

# ПРИМЕР

```
<?xml version=„1.0“ encoding=„UTF-8“?>  
<name nickname=„MrX“>  
    <first>Ivan</first>  
    <middle/>  
    <last>Ivanov</last>  
</name>
```

# ПРАВИЛА ЗА XML ДЕКЛАРАЦИИ:

- Започват с `<?xml`
- Завършват с `?>`
- Version – задължителен
- Encoding – опционален
- Редът на задаване - задължителен

# ENCODING

Задава знаковата кодировка, която се използва – в случая UTF-8 (вид Unicode)

# ИНСТРУКЦИИ ЗА ОБРАБОТКА (PROCESSING INSTRUCTIONS):

Можем да вградим инструкции, които  
показват как да се обработи информацията

# ПРИМЕР

```
<?xml version="1.0" encoding="UTF-16" standalone="yes"?>  
<name nickname="MrX">  
  <first>Ivan</first>  
  <middle/>  
  <?nameprocessor SELECT * FROM yyy?>  
  <last>Ivanov</last>  
</name>
```

# ИНСТРУКЦИИ ЗА ОБРАБОТКА (PROCESSING INSTRUCTIONS):

- <? - следван от името на приложението, което трябва да получи инструкцията (PITarget)
- След това се дава произволна инструкция (SELECT \* FROM ууу)
- Завършва с ?>